

MODELO MATEMÁTICO E ALGORITMO MEMÉTICO PARA UMA VARIANTE DO PROBLEMA DO CAIXEIRO ALUGADOR

Francisco Rodrigues Lima

UFRN

Campus Universitário Lagoa Nova, Natal - RN, 59078-900

francisco.lima.263@ufrn.edu.br

Matheus da Silva Menezes

UFRN

Campus Universitário Lagoa Nova, Natal - RN, 59078-900

matheus.menezes@ufrn.br

Sílvia Maria Diniz Monteiro Maia

UFRN

Campus Universitário Lagoa Nova, Natal - RN, 59078-900

silvia@dimap.ufrn.br

Ivan Mezzomo

UFERSA

R. Francisco Mota, 572 - Pres. Costa e Silva, Mossoró - RN

stefeson@ufersa.edu.br

Gustavo de Araújo Sabry

IFPB

R. Pres. Tancredo Neves, s/n - Jardim Sorrilândia, Sousa - PB

guga_sabry@hotmail.com

RESUMO

O Problema do Caixeiro Alugador ou *The Traveling Car Renter Problem (CaRS)* é uma generalização do Problema do Caixeiro Viajante (PCV) onde o percurso pode ser realizado utilizando diferentes carros alugados, também caracterizado como NP-Difícil. Todos os trabalhos relatados na literatura abordaram o problema sob a mesma ótica, onde o mesmo carro não pode ser alugado mais de uma vez, o carro alugado pode ser devolvido em qualquer cidade e existe uma cidade pré-definida para o início e final do percurso, denominada cidade base. Este artigo analisa o problema através da modificação destas características. O modelo matemático do problema é ajustado para considerar essa variação e é implementado em um *solver*, produzindo resultados para cinquenta e oito instâncias do problema. Um algoritmo memético é proposto, com buscas locais adequadas à variação do problema. O desempenho computacional do algoritmo memético proposto é comparado com a performance do *solver*.

PALAVRAS CHAVE. Problema do Caixeiro Alugador, Metaheurísticas, Programação Inteira Mista, Algoritmo Memético.

Tópicos (indique, em ordem de PRIORIDADE, o(s) tópico(s) de seu artigo): MH – Metaheurísticas, L&T – Logística e Transportes.

ABSTRACT

The Traveling Car Renter Problem (CaRS) is a generalization of the Traveling Salesman Problem (TSP) where the route can be completed using different rental cars, also characterized as NP-Hard. All the works reported in the literature approached the problem from the same perspective, where the same car cannot be rented more than once, the rental car can be returned in any city and there is a predefined city for the start and end of the route, called the base city. This paper analyzes the problem by modifying these characteristics. The mathematical model of the problem is adjusted to consider this variation and is implemented in a *solver*, producing results for fifty-eight instances of the problem. A memetic algorithm is proposed, with local searches appropriate to the variation of the problem. The computational performance of the proposed memetic algorithm is compared with the performance of the *solver*.

KEYWORDS. Traveling Car Renter Problem. Metaheuristics. Mixed Integer Programming. Memetic Algorithm.

Paper topics (indicate in order of PRIORITY the paper topic(s)): MH – Metaheuristics , L&T – Logistic and Transport.

1. Introdução

Este artigo aborda uma variação para o Problema do Caixeiro Alugador (CaRS), problema este introduzido em [Goldbarg et al., 2012]. O problema CaRS é uma generalização do Problema do Caixeiro Viajante (PCV) que tem como objetivo minimizar os custos de percorrer n cidades, cujo trajeto pode ser realizado utilizando carros alugados. Desde que o PCV é NP-Difícil, [Gutin e Punnen, 2002] e é um caso especial do CaRS, quando apenas um carro é utilizado no percurso, então o CaRS é também classificado como NP-Difícil.

O mercado de aluguel de veículos tem se consolidado como uma peça-chave no setor de mobilidade urbana e logística, oferecendo soluções flexíveis e sob demanda para indivíduos e empresas. Com o avanço das tecnologias digitais e o crescimento do modelo de consumo baseado em acesso ao invés de posse, o aluguel de carros tornou-se uma alternativa estratégica frente aos custos e à responsabilidade associados à propriedade de veículos. De acordo com um relatório da Mordor Intelligence (2024), espera-se que o mercado global de aluguel de carros cresça a uma taxa composta anual de aproximadamente 7% entre 2024 e 2029, impulsionado principalmente pelo turismo, pelo crescimento da economia colaborativa e pela demanda por serviços sob demanda em áreas urbanas densamente povoadas [Mordor Intelligence, 2024].

No problema geral do CaRS, existem diversas opções de locação de carros, a partir de diferentes empresas, disponíveis em cada cidade. Carros diferentes possuem custos diferentes para percorrer a rota. Estes custos podem ser considerados como o custo de locação, consumo de combustível e pagamento de pedágios. Além disso, se um carro é locado em uma cidade e devolvido em uma diferente, o usuário precisa considerar o pagamento de uma taxa de retorno para devolução do veículo para sua cidade de origem. O problema consiste ainda em visitar um conjunto de cidades, começando e terminando o *tour* no mesmo local, visitando todas as cidades e minimizando o custo de percorrer o caminho usando os carros locados.

O problema CaRS foi objeto de estudo em algumas pesquisas recentes, onde foram consideradas diferentes abordagens para resolução do problema.

Em termos de estratégias de solução exata, em [Goldbarg et al., 2012] foi implementado um *backtracking* para resolver o problema, cujo tempo de execução variou de 1 segundo para encontrar a solução ótima de um problema com dez cidades e dois carros disponíveis para locação, até 88 dias para encontrar a solução de um problema com dezesseis cidades e dois carros para locação, mostrando-se ineficiente em termos de encontrar resultados em um tempo computacional razoável. No trabalho de [Goldbarg et al., 2013] foi apresentado um modelo de programação quadrática com variáveis inteiras e implementado em um *solver*, com tempo de processamento limitado a 70.000 segundos, encontrando a solução ótima para dezoito pequenas instâncias. Destacamos que apesar da correteza dos resultados, o modelo apresentado no artigo estava incompleto. Nesse sentido, o trabalho de [da Silva e Ochi, 2016], apresentou uma tentativa de reformulação do modelo, porém este estava incorreto. A correção das formulações se deu em [Rios et al., 2021], destacando os erros da proposta de [da Silva e Ochi, 2016]. No trabalho de [Goldbarg et al., 2017] foram propostos três modelos para o CaRS, executados em três *solvers* distintos. Dois dos modelos apresentados possuem função objetivo quadrática e um modelo com restrição quadrática, apresentando também soluções de linearização para estes casos. Finalmente, em [Lacroix et al., 2021] foi desenvolvido um modelo baseado em coloração de vértices, apresentando novos limites para o problema.

Já em relação às abordagens metaheurísticas, foi proposto em [Goldbarg et al., 2012] um algoritmo GRASP (Greedy Randomized Search Procedure) com VNS (Variable Neighborhood Search) e um algoritmo memético, encontrando limites superiores para quarenta instâncias do CaRS. O algoritmo memético apresentou desempenho superior ao GRASP. Em [Goldbarg et al., 2013] foram utilizados um algoritmo memético e um algoritmo transgenético para resolver sessenta instâncias do

problema, com o algoritmo transgenético obtendo melhor performance do que o algoritmo memético. No trabalho de [da Silva e Ochi, 2016], são apresentados um algoritmo evolucionário híbrido e o um método híbrido denominado *Adaptive Local Search Procedure* (ALSP), apresentando bons resultados para instâncias não euclidianas. Em [Felipe et al., 2014] são utilizados Algoritmos Científicos *Scientific Algorithms* uma nova metaheurística inspirada no processo de pesquisa científica, e adaptado ao problema CaRS, obtendo bom desempenho tanto em tempo de execução quanto em qualidade das soluções encontradas.

Uma variante do problema CaRS, chamada Problema do Caixeiro Alugador com Coleta de Bônus ou *Prize Collecting Traveling Car Renter Problem* (*p-CaRS*), foi apresentada em [da Silva Menezes et al., 2014]. Nesta variação do problema, a cada cidade é associado um valor de bônus (ou satisfação presumida de visitação). O objetivo é determinar um ciclo que garanta a visitação de alguns vértices, coletando, pelo menos, um bônus mínimo pré-definido e minimizando o custo de percorrer o trajeto com os diferentes veículos alugados. Uma formulação matemática para esta variação do problema é apresentada e implementada em um *solver* produzindo resultados para sessenta e quatro instâncias. Também é proposto um algoritmo memético cujo desempenho é superior aos resultados obtidos pelo *solver*, tanto em relação ao tempo de processamento, quanto em relação à qualidade de solução.

Outra variante, proposta em [de Araujo Sabry et al., 2020], é denominada como o Problema do Caixeiro Alugador com Passageiros *Traveling Car Renter with Passengers* (*CaRSP*), onde, durante o tour do caixeiro, os passageiros podem pegar uma carona no carro do caixeiro e dividir as despesas da viagem com o motorista. Os passageiros têm diferentes pontos de embarque e desembarque e dividem os custos da parte da viagem em que estão no veículo. São apresentadas duas formulações de programação inteira mista para o CaRSP.

A maioria dos estudos existentes na literatura aborda uma única variante do problema CaRS, na qual o veículo alugado pode ser devolvido em qualquer cidade (devolução irrestrita), não é permitido alugar o mesmo tipo de carro mais de uma vez no trajeto (sem repetição) e o percurso deve começar e terminar em uma cidade fixa, denominada cidade base. Este artigo propõe uma nova variante do problema CaRS, mais alinhada à realidade prática, onde tais restrições são relaxadas: não há exigência de cidade base, o mesmo tipo de carro pode ser alugado múltiplas vezes ao longo do trajeto, e a devolução dos veículos é permitida apenas em um subconjunto específico de cidades. É apresentada uma formulação matemática para esta nova versão, implementada com o auxílio do *solver* GLPK. Além disso, propõe-se um algoritmo memético para a resolução do problema, cuja performance é avaliada em 58 instâncias, com os resultados comparados àqueles obtidos pelo *solver* exato.

Além desta introdução, o presente artigo está assim organizado: a seção 2 contempla a definição do problema do caixeiro alugador; na seção 3 temos a apresentação do algoritmo memético proposto, enquanto a seção 4 contempla a metodologia dos experimentos que serão realizados. Os resultados e discussões são contemplados na seção 5. Finalmente, as considerações finais são apresentadas na seção 6.

2. Uma Variante do Problema do Caixeiro Alugador

Na literatura, todos os trabalhos relatados investigam a mesma variante do problema CaRS, tal como foi proposto em [Goldbarg et al., 2012]. A alternativa de devolução do carro é irrestrita, onde o carro locado pode ser devolvido em qualquer cidade. A integridade do contrato não permite que o mesmo tipo de carro seja alugado mais que uma vez durante o trajeto, chamada de sem repetição. Finalmente, o modelo define a cidade base do caixeiro, uma cidade fixa, onde o percurso deve começar e terminar (por convenção do problema, é o primeiro vértice da instância).

Este artigo apresenta um cenário diferente para o problema CaRS, onde essas condições

são modificadas, buscando um cenário alternativo para o problema. Na prática, o usuário pode não ter uma cidade fixa para o início do trajeto, as empresas de aluguel não necessariamente aplicam restrições sobre alugar o mesmo tipo de carro e não temos garantia de que todas as cidades estão aptas a receber a devolução de veículos.

Neste sentido, a versão em estudo para o problema, pressupõe que o mesmo tipo de carro pode ser locado mais de uma vez durante o trajeto, não existe uma cidade base pré-definida e são aplicadas restrições de devolução do carro alugado, onde o mesmo só pode ser devolvido em algumas cidades pré-definidas do percurso.

Seja $G = (V, A)$ um grafo completo onde V é um conjunto com n nós (cidades) e A é um conjunto de arcos (estradas ligando as cidades). Neste problema, C representa o conjunto de carros. Todos os carros estão disponíveis para aluguel em todas as cidades. O custo para o carro $c \in C$ percorrer a aresta $(i, j) \in A$ é denotado por d_{ij}^c . Se um carro $c \in C$ é locado em uma cidade i e devolvido em uma cidade j , $i \neq j$, uma taxa para retornar o carro c de volta para a cidade i , γ_{ij}^c deve ser considerada. Além disso, nesta versão do problema, um carro locado em uma cidade i e retornado em uma cidade j , $i \neq j$ pode ser locado novamente em outra parte do trajeto. Em relação à restrição de devolução, de forma a aproveitar as instâncias existentes sem a necessidade de modificações, foi considerado que os carros só podem ser devolvidos em cidades que possuam índice ímpar. A formulação matemática considera ainda as seguintes variáveis binárias: f_{ij}^c com valor 1 quando o carro c percorre a aresta (i, j) de i para j e 0 caso contrário; w_{ij}^c com valor 1 quando o carro c é alugado em uma cidade j e devolvido em uma cidade i ; a_i^c com valor 1 quando o carro c é locado em uma cidade i ; e_i^c com valor 1 quando o carro c é devolvido na cidade i .

A seguir é apresentado um modelo para a variante do problema CaRS, baseado na formulação DFJ para o CaRS apresentada em [Goldbarg et al., 2017]:

$$\min \sum_{c \in C} \sum_{i, j \in V} f_{ij}^c d_{ij}^c + \sum_{c \in C} \sum_{i, j \in V} \gamma_{ij}^c w_{ij}^c \quad (1)$$

$$\sum_{c \in C} \sum_{j \in V} f_{ij}^c = 1 \quad \forall i \in V \quad (2)$$

$$\sum_{c \in C} \sum_{i \in V} f_{ij}^c = 1 \quad \forall j \in V \quad (3)$$

$$a_i^c = \left(\sum_{j \in v=V} f_{ij}^c \right) \left(\sum_{x \in C, x \neq c} \sum_{h \in V} f_{hi}^x \right) \quad \forall c \in C, i \in V \quad (4)$$

$$e_i^c = \left(\sum_{j \in v=V} f_{ji}^c \right) \left(\sum_{x \in C, x \neq c} \sum_{h \in V} f_{ih}^x \right) \quad \forall c \in C, i \in V \quad (5)$$

$$w_{ij}^c = a_j^c e_i^c \quad \forall c \in C, i \in V \quad (6)$$

$$\sum_{c \in C} \sum_{i \in V} a_i^c \geq 1 \quad (7)$$

$$2 \leq u_i \leq n \quad \forall i = 2, \dots, n \quad (8)$$

$$u_i - u_j + 1 \leq (n-1) \left(1 - \sum_{c \in C} f_{ij}^c \right) \quad i, j = 2, \dots, n \quad (9)$$

$$f_{ij}^c, d_{ij}^c, a_{ij}^c, e_{ij}^c \in 0, 1 \quad (10)$$

$$v_i \in \mathbb{N} \quad (11)$$

A equação (1) representa a função objetivo. Ela considera o custo acumulado de percorrer as arestas com diferentes carros e os seus respectivos custos relativos ao retorno. As restrições (2) e (3) mostram que cada vertice é visitado uma vez e que se um carro chega em um vértice i então

um carro deve deixar aquele vértice. As restrições (4) e (5) controlam o aluguel e a devolução do veículo, e foram propostas em Goldberg et al. [2017]. Dessa forma, a restrição (4) registra se o carro c foi locado no nó i enquanto a restrição (5) registra se o carro c foi devolvido no nó i . A restrição (6) registra se um carro c foi locado em cidade i e devolvido em uma cidade j . A restrição (7) assegura que pelo menos um carro é locado. Restrições (8), (9) proíbem a formação de sub-rotas e foram adaptadas da formulação de Miller-Tucker-Zemlin (MTZ) para o PCV [Miller et al., 1960]. A restrição (10) assegura que as variáveis são binárias e a restrição (11) define que as variáveis u_i são inteiros positivos.

As restrições (4)-(6) são quadráticas e suas variáveis são binárias. Para contornar este problema, utilizamos a linearização usual [Liberti, 2007]. Uma restrição não linear do tipo apresentado em (12), é substituído pelo conjunto de equações (13), (14), (15), (16).

$$z = x \times y \quad (12)$$

$$z \leq x \quad (13)$$

$$z \leq y \quad (14)$$

$$z \geq x + y - 1 \quad (15)$$

$$z, x, y \in [0, 1] \quad (16)$$

Esta linearização foi implementada no GLPK conforme sugerido em [AIMMS, 2012].

3. Algoritmo Memético Proposto

Os algoritmos meméticos foram propostos em [Glover e Woolsey, 1974] e tem sido utilizados com sucesso em uma variada gama de aplicações [AIMMS, 2012], incluindo problemas de roteamento. O algoritmo memético proposto para esta versão do CaRS é baseado em [da Silva Menezes et al., 2014] e apresentado no algoritmo 1. Ele recebe como parâmetros de entrada o nome da instância *inst*, o tamanho da população *tamPop*, a taxa de recombinação *#recombina*, e o percentual da população que deverá compor a elite *#elite*.

Algorithm 1: Pseudo-código do algoritmo memético proposto

```

begin
  main(inst, tamPop, #recombina, #elite, limitIter)
  leInstancia(inst)
  pop[] ← geraPopIni(tamPop)
  numIter ← 0
  buscaLocalMultiOper(pop)
  elitePop[] ← geraPopElite(pop, #elite)
  while numIter ++ < limitIter do
    off[] ← recombina(pop, elitePop[], #recombina)
    buscaLocalMultiOper(off)
    pop[] ← binaryTournament(pop, off)
    elitePop[] ← geraPopElite(pop, #elite)
  end
end

```

Uma representação gráfica para o cromossomo pode ser vista na Figura 1, composta por um vetor bi-dimensional, onde o percurso é representado em uma dimensão (superior), e a sequência de carros utilizada é representado na outra (inferior). Nesta figura, o cromossomo representa

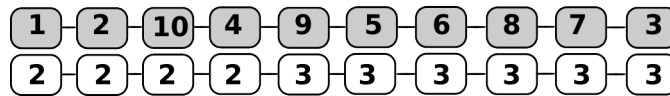


Figura 1: Representação Gráfica do Cromossomo

uma solução onde o carro 2 é contratado na cidade base e entregue na cidade 9, onde o carro 3 é contratado e assim por diante.

A população inicial é gerada a partir do procedimento *geraPopIni()*, que recebe o tamanho da população *tamPop* como argumento de entrada. Uma heurística gulosa, adaptada de [da Silva Menezes et al., 2014] é usada para gerar os cromossomos da população inicial. A cidade inicial i e um carro c_1 são escolhidos aleatoriamente. A cidade j onde o carro c_1 será devolvido é escolhida aleatoriamente. Um caminho entre as cidades i e j é construído, aplicando a heurística do vizinho mais próximo, considerando os custos de percurso associados ao carro c_1 . A cidade j é a cidade inicial da próxima parte da rota e um novo carro c_2 e uma nova cidade z são sorteados aleatoriamente para esta parte do percurso. Claramente, a cidade z não pode ser nenhuma cidade já visitada no caminho. Um caminho entre as cidades j e z é construído com as cidades não visitadas. O procedimento continua selecionando outra cidade e carro não utilizados enquanto houver cidades a serem visitadas.

O maior desafio dos métodos heurísticos que utilizam busca local é definir estratégias eficientes para cobrir o espaço de soluções, explorando regiões promissoras [Ong et al., 2010]. Nesta versão do problema CaRS, temos duas áreas principais de atuação da busca local: a melhor rota e melhor ponto de troca dos veículos. A fase de busca local proposta no algoritmo busca atuar de forma consistente nestas áreas, de forma a melhorar as soluções candidatas. A fase de busca local é baseada em [Goldbarg et al., 2012; da Silva Menezes et al., 2014] e é executada no procedimento *buscaLocalMultiOper*. Para esta busca, efetuamos novas adaptações nos operadores, procedimentos de reparação e ajustes nos procedimentos para explorar a vizinhança da variação do problema em estudo, conforme descrito a seguir:

- *invertSol*: Este método inverte a ordem de visitação das cidades na solução candidata. Os mesmos carros são associados às mesmas cidades, porém os pontos de aluguel e devolução são trocados. Por exemplo, considere o caminho (1, 2, 3, 4, 5) contendo cinco cidades, onde o carro 1 foi locado na cidade 1 e devolvido na cidade 3 e o carro 2 foi locado na cidade 3 e devolvido na cidade 1. Após a aplicação do método, o percurso se torna (5,4,3,2,1) com o carro 1 locado na cidade 5 e devolvido na cidade 3 e o carro 2 sendo locado na cidade 3 e devolvido na cidade 5.
- 2-opt: é uma busca local simples, proposta em [Croes, 1958] para o PCV. Um movimento 2-opt consiste em eliminar duas arestas da solução e ligar novamente essas duas arestas de forma cruzada. Se esta nova configuração for melhor que a anterior, mantenha a nova rota. Caso contrário, escolha novamente duas arestas para análise.
- *replaceSavingCar*: Este procedimento, adotado em [da Silva Menezes et al., 2014], tem como foco a inserção de um novo carro, substituindo um que já esteja na solução, se possível.

Essas três buscas locais são aplicadas em sequência no procedimento *buscaLocalMultiOper()*. Se a solução de entrada, representada pelo indivíduo *ind* é melhorada durante a busca local, então o *ind* é substituído pelo melhorado.

De maneira similar a [da Silva Menezes et al., 2014], uma população de elite é considerada para a recombinação. Esta população de elite, *elitepop*, contém os $\#elite$ percentuais melhores indivíduos da população atual em termos de seus valores de aptidão, calculado de acordo com a expressão (1).

O procedimento de recombinação, *recombina()*, é baseado no *Crossover IX* utilizado em [da Silva Menezes et al., 2014]. Um dos pais é escolhido da população geral e o outro é escolhido na população de elite. Ambos são escolhidos aleatoriamente.

Assim como nas versões anteriores, pode ser necessário restaurar a viabilidade de soluções geradas durante o processo de recombinação ou de busca local. Nesta versão do CaRS, a inviabilidade pode ocorrer na repetição de cidades na rota ou na devolução do carro. O procedimento de restauração da rota é o mesmo utilizado em [Goldbarg et al., 2012]. A restauração devido a devolução do veículo é feita através da substituição do carro que está sendo devolvido em cidade de índice par pelo carro que está associado à cidade imediatamente posterior. O procedimento continua até que o índice seja ímpar. Caso o procedimento não consiga restaurar o cromossomo, então o carro associado à cidade inicial é utilizado em todo o percurso.

4. Metodologia

Para verificar a efetividade do algoritmo proposto, um estudo empírico com os resultados de *benchmark* do CaRS, é conduzido e apresentado nessa seção. A base de dados, disponível em <http://www.dimap.ufrn.br/lae/en/projects/CaRS.php>, contém a descrição de cada grupo de instâncias e os formatos de arquivo.

Nesta versão do problema, as instâncias consideram as seguintes características: todos os carros podem ser locados em todas as cidades; os carros só podem ser devolvidos em cidades com índice ímpar; o mesmo tipo de carro pode ser locado mais de uma vez durante o trajeto; a taxa de retorno para devolução de veículo para sua cidade de aluguel não é associada à topologia do problema; o custo de percurso é simétrico; o grafo é completo. As instâncias são divididas em duas classes: Euclidianos e não-Euclidianos

A formulação matemática apresentada na seção 2 foi implementada no *solver* GLPK, versão 4.55., com tempo de processamento limitado em até 10.000 segundos. O processamento é encerrado caso o *solver* encontre a solução ótima ou atinja o tempo de processamento pré-definido.

Os parâmetros do algoritmo memético foram os mesmos utilizados em [da Silva Menezes et al., 2014] e são: $tamPop = 150$, $\#recombina = 0.4$ e $\#elite = 0.3$. O critério de parada foi de $limitIter = 100$ iterações.

Para o algoritmo memético, cada instância foi executada trinta vezes. A partir destes resultados foram computados o tempo de processamento médio, o menor valor encontrado para a função objetivo, o valor médio da função objetivo, e a quantidade de vezes que a melhor solução foi encontrada.

O algoritmo foi executado em um notebook Dell com processador Intel® Core™ i5 de 13 geração, 16 GB de memória RAM e SSD de 512 GB. O sistema operacional foi o Ubuntu 24.04 LTS. O algoritmo memético foi desenvolvido em linguagem C++, e compilador GCC 13.3.00.

5. Resultados e Discussões

Os resultados do experimento computacional está disponível nas tabelas 1 e 2. As colunas *Nome*, *nCid* e *nCar* são referentes às características das instâncias e são, respectivamente, a identificação da instância, o número de cidades e o número de carros. As colunas *Min* and *T(s)*, relativas ao *solver* GLPK, mostram o valor da melhor solução viável encontrado e o tempo de processamento em segundos. As colunas *Min*, *Media*, *F* and *T(s)*, relativos ao algoritmo memético mostram, respectivamente, o valor da solução mínima, a média das soluções mínimas encontradas em trinta rodadas do algoritmo, o número de vezes que a melhor solução foi encontrada e o tempo

de processamento em segundos. A coluna *Gap*, para cada estratégia, apresenta o desvio percentual do valor da coluna *Min* em relação ao limite inferior para a solução exata, calculado pelo GLPK.

A tabela 1 mostra que o GLPK encontra a solução ótima para quase todas as instâncias Euclidianas. A exceção fica para as instâncias *China17e*, *Canada17e*, *Cazaquistao15e* e *Brasil16e* nas quais o GLPK parou por limitação de tempo. O desvio percentual em relação ao limite inferior calculado pelo GLPK para essas instâncias foi de 9.90, 2.30, 7.50 and 3.5, respectivamente. GLPK precisou de um tempo maior para encontrar a solução das instâncias em relação ao algoritmo memético. Esta diferença é maior, a medida em que o tamanho da instância cresce. O GLPK encontra soluções melhores que o algoritmo memético em 3 instancias e o oposto ocorre em 1 instância. O algoritmo memético utilizou um tempo médio de processamento menor do que o GLPK. Em média, o GLPK utilizou 2413.69 segundos para encontrar as soluções para as instâncias Euclidianas, enquanto o algoritmo memético utilizou 5.29 segundos. A frequência média em que o algoritmo memético encontrou a melhor solução foi de 16.34.

A tabela 2 mostra que o GLPK não conseguiu encontrar a solução ótima para dezessete instâncias não Euclidianas, onde o algoritmo parou devido a limitação de tempo. O desvio percentual variou entre 6.30 e 34.40. Em quinze dessas dezessete instâncias, o algoritmo memético encontrou melhores soluções que as produzidas pelo GLPK. O algoritmo memético também utilizou, em média, menos tempo de processamento para obter estas soluções. O algoritmo memético utilizou, em média, de 2.01 até 10.03 segundos para processar as instâncias não Euclidianas. Os resultados do GLPK foram obtidos em tempos que variaram entre 8 até 10000 segundos. A frequência média em que o algoritmo memético encontrou a melhor solução foi de 13.24 para as instâncias não Euclidianas.

Tabela 1: Resultados para as Instâncias Euclidianas

Instancias			Exato (GLPK)			Memetico			
Nome	nCid	nCar	Min	T(s)	GAP (%)	Min	Media	F	T(s)
Mauritania10e	10	2	622	7	0	622	622	30	2.01
Colombia11e	11	2	620	3	0	620	620.13	29	2.50
Angola12e	12	2	714	11	0	714	714	30	3.03
Peru13e	13	2	672	5	0	672	672	30	3.54
Libia14e	14	2	754	46	0	754	756.1	21	4.07
BrasilRJ14e	14	2	305	463	0	305	305.37	19	4.19
Congo15e	15	2	756	7	0	756	756.4	27	4.88
Argentina16e	16	2	959	526	0	959	964.17	19	5.63
EUA17e	17	2	950	1879	0	950	971.67	2	6.42
Bolivia10e	10	3	739	33	0	739	740.9	15	2.51
AfricaSul11e	11	3	679	18	0	679	679	30	3.00
Niger12e	12	3	854	90	0	854	854	30	3.58
Mongolia13e	13	3	764	526	0	764	764.9	27	4.28
Indonesial4e	14	3	895	303	0	895	899.77	16	5.04
Argelia15e	15	3	840	2619	0	840	851.17	6	5.81
India16e	16	3	1136	1384	0	1136	1163.93	1	6.75
China17e	17	3	1008	10000	9.9	1011	1026.4	1	7.86
Etiopia10e	10	4	631	53	0	631	631.03	29	2.85
Mali11e	11	4	673	60	0	673	680.47	20	3.49
Chade12e	12	4	859	232	0	859	867.13	15	4.19
Ira13e	13	4	996	2781	0	1051	1055.2	6	4.99
Mexico14e	14	4	843	1619	0	843	851.23	15	5.87
Sudao15e	15	4	863	2802	0	863	871.1	9	6.79
Australia16e	16	4	1051	3738	0	1051	1054.9	4	7.79
Canada17e	17	4	1280	10000	2.3	1280	1337.77	8	8.98
Arabia14e	14	5	987	2905	0	987	997.17	12	6.58
Cazaquistao15e	15	5	987	10000	7.5	966	981.63	9	7.68
Brasil16e	16	5	1216	10000	3.5	1216	1232.83	13	8.88
Russia17e	17	5	1046	7887	0	1057	1092	1	10.09

Tabela 2: Resultados para as Instâncias não-Euclidianas

Instancia			Exato (GLPK)			Memetico			
Nome	nCid	nCar	Min	T(s)	GAP (%)	Min	Media	F	T(s)
Mauritania10n	10	2	612	8	0	612	612	30	1.97
Colombia11n	11	2	639	140	0	639	639.1	28	2.49
Angola12n	12	2	671	126	0	671	671.1	27	2.93
Peru13n	13	2	707	2453	0	707	707.33	26	3.46
Libia14n	14	2	745	10000	7.8	744	744.17	27	4.14
BrasilRJ14n	14	2	168	10000	10.1	167	167.37	19	4.20
Congo15n	15	2	886	1745	0	886	886.07	28	4.79
Argentina16n	16	2	878	10000	13.9	876	877.03	12	5.61
EUA17n	17	2	822	10000	8.9	822	824.43	2	6.36
Bolivia10n	10	3	825	31	0	825	826	24	2.46
AfricaSul11n	11	3	781	3040	0	781	781.93	19	2.99
Niger12n	12	3	826	1403	0	826	828.87	12	3.54
Mongolia13n	13	3	804	10000	13.2	799	804.97	8	4.16
Indonesial4n	14	3	848	10000	14.5	843	850.3	13	4.95
Argelia15n	15	3	865	10000	20.8	863	868.83	2	5.76
India16n	16	3	1052	10000	16.7	1050	1064.27	10	6.64
China17n	17	3	947	10000	15.9	945	953.63	1	7.67
Etiopia10n	10	4	719	889	0	719	720.23	12	2.85
Mali11n	11	4	841	5879	0	841	841.63	20	3.42
Chade12n	12	4	891	8161	0	891	982.67	18	4.21
Ira13n	13	4	1010	10000	19.5	989	1028.77	7	4.95
Mexico14n	14	4	959	10000	28.3	950	965.37	4	5.74
Sudao15n	15	4	1099	10000	27.7	1096	1096.7	19	6.63
Australia16n	16	4	1116	10000	21.2	1101	1114.73	4	7.66
Canada17n	17	4	1289	10000	28.5	1277	1305.27	4	8.86
Arabia14n	14	5	1129	10000	6.3	1129	1175.83	2	6.58
Cazaquistao15n	15	5	1139	10000	24.1	1134	1161.57	3	7.58
Brasil16n	16	5	1320	10000	34.4	1266	1335	2	8.85
Russia17n	17	5	1133	10000	29.6	1104	1136.5	1	10.03

6. Considerações Finais

Este trabalho apresentou uma nova variante do problema CaRS [Goldberg et al., 2012], caracterizada pela ausência de uma cidade fixa de partida, pela possibilidade de alugar o mesmo tipo de carro mais de uma vez ao longo do trajeto e pela introdução de restrições de devolução de veículos, limitando os pontos de entrega a um subconjunto de cidades. Foi proposta uma formulação matemática para essa versão, cuja implementação foi realizada no *solver* GLPK, com tempo de processamento limitado a 10.000 segundos. Além disso, desenvolveu-se um algoritmo memético com três estratégias de busca local, cuja eficácia foi avaliada por meio de experimentos computacionais.

O desempenho do algoritmo foi comparado ao do GLPK em um conjunto de 58 instâncias, sendo 29 Euclidianas e 29 não Euclidianas. O GLPK encontrou soluções exatas em 37 instâncias, enquanto o algoritmo memético estabeleceu novos limites em outras 17. Nas instâncias Euclidianas, o algoritmo heurístico obteve soluções próximas às exatas em quase todos os casos, superando o GLPK em uma instância. Já nas instâncias não Euclidianas, o algoritmo memético apresentou desempenho superior em 16 casos. Os resultados indicam que a proposta é particularmente eficaz para instâncias de maior dimensão e topologias mais complexas, além de apresentar menor tempo de processamento em comparação com o GLPK, reforçando seu potencial como abordagem alternativa para a resolução desta variante do problema CaRS em contextos práticos.

Como esta variação do problema CaRS é nova na literatura, várias inovações podem ser implementadas em pesquisas futuras, tais como: (a) desenvolver e analisar outras metaheurísticas adaptadas às características dessa versão do problema; (b) desenvolver e investigar outras buscas locais adaptadas ao problema; (c) desenvolver algoritmos para outras variações do problema CaRS.

Referências

- AIMMS (2012). Integer programming tricks. URL <http://www.aimms.com>.
- Croes, G. A. (1958). A method for solving traveling-salesman problems. *Operations research*, 6 (6):791–812.
- da Silva, A. R. V. e Ochi, L. S. (2016). An efficient hybrid algorithm for the traveling car renter problem. *Expert Systems with Applications*, 64:132–140. ISSN 0957-4174. URL <https://www.sciencedirect.com/science/article/pii/S0957417416303852>.
- da Silva Menezes, M., Goldberg, M. C., e Goldberg, E. F. (2014). A memetic algorithm for the prize-collecting traveling car renter problem. In *Evolutionary Computation (CEC), 2014 IEEE Congress on*, p. 3258–3265. IEEE.
- de Araujo Sabry, G., Goldberg, M. C., Goldberg, E. F. G., da Silva Menezes, M., e Calheiros, Z. S. A. (2020). Models and linearizations for the traveling car renter with passengers. *SpringerNature Complete Journals*, 15(1):59–81.
- Felipe, D., Ferreira Gouvêa Goldberg, E., e Goldberg, M. C. (2014). Scientific algorithms for the car renter salesman problem. In *2014 IEEE Congress on Evolutionary Computation (CEC)*, p. 873–879.
- Glover, F. e Woolsey, E. (1974). converting the 0-1 polynomial programming problem to a 0-1 linear program. *Operations research*, 22(1):180–182.
- Goldberg, M. C., Goldberg, E. F. G., Luna, H. P. L., Menezes, M. S., e Corrales, L. (2017). Integer programming models and linearizations for the traveling car renter problem. *SpringerNature Complete Journals*, 12(4):743–761.
- Goldberg, M. C., Goldberg, E. F., Asconavieta, P. H., Menezes, M. d. S., e Luna, H. P. (2013). A transgenetic algorithm applied to the traveling car renter problem. *Expert Systems with Applications*, 40(16):6298–6310.
- Goldberg, M. C., Asconavieta, P. H., e Goldberg, E. F. G. (2012). Memetic algorithm for the traveling car renter problem: an experimental investigation. *Memetic Computing*, 4(2):89–108.
- Gutin, G. e Punnen, A. P. (2002). *The traveling salesman problem and its variations*, volume 12. Springer Science & Business Media.
- Lacroix, M., Solis, Y. R., e Calvo, R. W. (2021). Efficient formulations for the traveling car renter problem and its quota variant. *SpringerNature Complete Journals*, 15(6):1905–1930.
- Liberti, L. (2007). Compact linearization for binary quadratic problems. *4OR*, 5(3):231–245.
- Miller, C. E., Tucker, A. W., e Zemlin, R. A. (1960). Integer programming formulation of traveling salesman problems. *Journal of the ACM (JACM)*, 7(4):326–329.
- Mordor Intelligence (2024). Car rental market - growth, trends, covid-19 impact, and forecasts (2024 - 2029). <https://www.mordorintelligence.com/industry-reports/car-rental-market>. Accessed: 2025-04-25.

- Ong, Y.-S., Lim, M. H., e Chen, X. (2010). Research frontier-memetic computation—past, present & future. *IEEE Computational Intelligence Magazine*, 5(2):24.
- Rios, B. H. O., Casquina, J. C., Añasco, H. H. V., e Paz-Valderrama, A. (2021). Corrected formulations for the traveling car renter problem. In Simos, D. E., Pardalos, P. M., e Kotsireas, I. S., editors, *Learning and Intelligent Optimization*, p. 351–363, Cham. Springer International Publishing. ISBN 978-3-030-92121-7.